

Entregable 7. Estrategia de entrega y gobierno técnico

7.1 Resumen

Practica	Estado
Estrategia de ramas	Aplicada en el repositorio
Revision de codigo	Via pull request
Analisis de seguridad	Parcial
Pruebas automatizadas	Pendiente
CI/CD	Implementado con GitHub Actions
Despliegue azul-verde	Propuesta
Rollback	Propuesta
Documentacion de arquitectura	Implementada
Proceso de aprobacion	Propuesta

7.2 Ramas

master siempre desplegable, protegida
develop integracion
feature/* una rama por area de trabajo

Aplicada en el repositorio: las ramas de trabajo se integran en develop con --no-ff para que la fusion quede visible en el grafo, y develop pasa a master por pull request.

Reglas sobre master: sin push directos, pull request obligatorio con una aprobacion y las comprobaciones automaticas en verde. Para produccion se anadirian ramas release para estabilizar antes de publicar y hotfix sobre master con reintegracion a develop.

7.3 Revision de codigo

Al menos una aprobacion. Que se mira, en este orden:

- **Correccion:** hace lo que dice, y los casos limite estan cubiertos.
- **Limites:** la logica de negocio esta en el dominio y no filtrada al controlador.
- **Seguridad:** toda consulta lleva entidadId, y no hay secretos en el diff.
- **Trazabilidad:** los cambios de estado dejan historial.
- **Estilo:** lo ultimo, y automatizable.

Un pull request grande no se revisa, se aprueba. De ahi que los commits sean pequeños.

7.4 Analisis de seguridad

Control	Herramienta	Cuando
Dependencias vulnerables	dotnet list package --vulnerable, npm audit	Cada push y pull request
Secretos versionados	Comprobacion propia en la integracion continua	Cada push y pull request
Analisis estatico	CodeQL o SonarQube	Propuesto
Contenedores	Trivy	Propuesto

La comprobacion de secretos es la unica implementada, y es la que mas importa aqui: un secreto en un commit sigue en el historial aunque se borre despues, y rotar la clave es la unica correccion real. El trabajo falla si appsettings.Development.json llega a versionarse o si una plantilla trae un valor real en lugar de un marcador.

7.5 Pruebas automatizadas

Pendientes. La piramide propuesta, priorizada por lo que mas aporta en este sistema:

Nivel	Que cubre
Unitarias del dominio	Maquina de estados: cada transicion legal y el rechazo de las ilegales
Unitarias de aplicacion	Segregacion por entidad, reproceso y reglas de negocio
Integracion	Outbox escrito en la misma transaccion, idempotencia del consumidor
Extremo a extremo	Los tres caminos por monto, hasta el estado final

El dominio es lo mas barato de probar y donde vive el riesgo: no tiene dependencias, asi que las pruebas no necesitan base de datos ni broker.

7.6 Integracion continua

Implementada en .github/workflows/ci.yml, con tres trabajos en paralelo:

Trabajo	Que hace
backend	Restaura, compila en Release, ejecuta pruebas y lista dependencias vulnerables
frontend	npm ci, compila el portal y ejecuta npm audit
secretos	Falla si se versiona configuracion local o si una plantilla trae un valor real

Se dispara en cada push a master, develop y ramas feature, y en cada pull request a master y develop.

La progresion completa de despliegue, aun no implementada:

```
push a feature/*    ->  compilar + pruebas + analisis
PR a develop        ->  lo anterior + cobertura + revision
merge a develop     ->  desplegar a integracion
PR a master         ->  aprobacion adicional
merge a master      ->  desplegar a produccion (azul-verde)
```

Puertas de calidad: la compilacion no puede tener advertencias, las pruebas deben pasar, sin dependencias con vulnerabilidad alta o critica y sin secretos detectados.

7.7 Despliegue

Azul-verde para la API y el portal: se levanta el entorno nuevo, se verifica con las comprobaciones de salud, se conmuta el trafico y el anterior queda intacto para revertir.

Los **workers** requieren un cuidado distinto porque consumen mensajes: se detiene el consumo, se espera a que terminen los mensajes en vuelo y luego se reemplazan. El outbox hace segura la interrupcion, ya que los eventos siguen en la base, y la idempotencia hace seguro el solapamiento si dos versiones conviven un momento.

Un despliegue canario tendria sentido con muchas entidades: enrutar una sola a la version nueva, verificar y extender. Con dos entidades no aporta.

7.8 Rollback

Elemento	Como se revierte
API y portal	Conmutar de vuelta al entorno anterior, en segundos
Workers	Volver a la imagen anterior; el outbox conserva lo pendiente
Esquema de base	Solo cambios compatibles hacia atras

La regla del esquema es la que hace posible todo lo demas: agregar columnas opcionales y tablas, nunca eliminar ni renombrar en el mismo despliegue que las usa. Eliminar se hace en un despliegue posterior, cuando ya nada las referencia. Si no, el rollback de la aplicacion deja una base incompatible y no hay vuelta atras.

7.9 Documentacion de arquitectura

Versionada junto al codigo, en el directorio docs del repositorio, para que envejezca con el.

Documento	Contenido
01-arquitectura	Componentes, zonas, flujos y puntos de fallo
02-mensajeria	Topologia, contratos, reintentos e idempotencia
03-consistencia	Outbox, saga y compensaciones
05-seguridad	Controles de seguridad
06-operacion	Objetivos de servicio, observabilidad y continuidad
07-entrega-gobierno	Este documento

Los diagramas estan escritos en Mermaid dentro de los propios archivos: se renderizan en el navegador y se revisan en el diff como texto. Un diagrama que solo existe como imagen deja de ser cierto en cuanto alguien cambia el codigo.

7.10 Proceso de aprobacion

Antes de autorizar un desarrollo, exigiria por escrito:

- **Que problema resuelve** y como se medira que lo resolvio.
- **Limites de contexto:** que componentes toca y quien es dueño de cada dato.
- **Contratos:** eventos y endpoints nuevos, con su estrategia de versionado.
- **Consistencia:** que pasa si falla a mitad, que se compensa y que no.
- **Modos de fallo:** que deja de funcionar si se cae cada dependencia.
- **Seguridad:** que datos sensibles toca y como se protegen.
- **Observabilidad:** que metricas diran si funciona en produccion.
- **Plan de reversion:** como se deshace si sale mal.

Los puntos de consistencia, modos de fallo y plan de reversion son los que mas se omiten y los que mas caro salen. Una propuesta que no sabe responder que ocurre si el banco no contesta no esta lista para desarrollarse, por bien dibujado que este el diagrama.